

# Objektorientierte Programmierung (OOP)

---

C++ · Klassen · Vererbung · Polymorphie · Prüfungsfragen

*Lernskript zur Vorbereitung auf die mündliche Prüfung*

## Inhaltsverzeichnis

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Teil A – Theorie (nach Kusche) .....                                                | 3  |
| 1. Warum objektorientierte Programmierung? .....                                    | 3  |
| 2. Die vier Grundpfeiler der OOP .....                                              | 3  |
| 3. Grundbegriffe .....                                                              | 3  |
| 4. Klassen und Objekte in C++ .....                                                 | 4  |
| 5. Konstruktoren (Ctor) .....                                                       | 4  |
| 6. Destruktoren (Dtor) .....                                                        | 4  |
| 7. Initialisierungsliste, this, const-Methoden .....                                | 5  |
| 8. Statische Member und Methoden .....                                              | 5  |
| 9. Heap vs. Stack .....                                                             | 5  |
| 10. Dynamische Speicherverwaltung: new und delete .....                             | 5  |
| 11. Vererbung .....                                                                 | 6  |
| 12. Überschreiben und virtuelle Methoden (Polymorphie) .....                        | 6  |
| 13. Abstrakte Klassen und rein virtuelle Methoden .....                             | 6  |
| 14. Typumwandlung zwischen verwandten Klassen .....                                 | 7  |
| 15. Aufteilung in Dateien .....                                                     | 7  |
| 16. Unterschied C und C++ .....                                                     | 7  |
| 17. Templates .....                                                                 | 7  |
| 18. Geschichte und Sprachen (kurz) .....                                            | 7  |
| Teil B – Prüfungsfragen mit Modellantworten .....                                   | 8  |
| F1: Erklären Sie OOP – was sind die Kernkonzepte / Säulen? .....                    | 8  |
| F2: Unterschied zwischen Klasse und Objekt? .....                                   | 8  |
| F3: Wie ist eine Klasse aufgebaut? (Lieblingsfrage) .....                           | 8  |
| F4: Konstruktor und Destruktor (besonders Destruktor) .....                         | 9  |
| F5: Erklären Sie Vererbung .....                                                    | 9  |
| F6: Was ist Aggregation? (und die anderen UML-Beziehungen) .....                    | 9  |
| F7: Zeichnen Sie ein Klassendiagramm .....                                          | 11 |
| F8: Schreiben Sie eine Klasse an (mit public, private, protected) .....             | 11 |
| F9: Unterschied C und C++ .....                                                     | 11 |
| F10: Was sind Templates? .....                                                      | 12 |
| F11: Heap vs. Stack .....                                                           | 12 |
| F12: Sechs Programmiersprachen – objektorientiert? Compiler oder Interpreter? ..... | 12 |
| F13: Wofür / warum benutzt man OOP überhaupt? .....                                 | 12 |
| Schnell-Check vor der Prüfung .....                                                 | 13 |

# Teil A – Theorie (nach Kusche)

---

## 1. Warum objektorientierte Programmierung?

OOP ist eine Weiterentwicklung der prozeduralen (ablauforientierten) Programmierung. Ziel ist es, große Software-Systeme – hunderttausende Zeilen, zehntausende Funktionen – schneller, fehlerfreier und übersichtlicher zu entwerfen und zu programmieren.

**Zentrale Beobachtung:** Es arbeiten nicht alle Funktionen mit allen Daten. Fast jede Funktion gehört zu einer ganz bestimmten Datenstruktur. Lösung: Man ordnet die Funktionen den Daten zu, auf denen sie arbeiten – eine Datenstruktur besteht aus den Daten und dem dazugehörigen Code.

**Drei Probleme werden gelöst:**

- **Strukturierung / Modularisierung:** Jede Datenstruktur bildet eine Einheit mit eigenem Namensraum; große Programme werden datenstrukturweise auf mehrere Programmierer verteilt.
- **Kapselung:** Interne Daten sind von außen nicht sichtbar; der Zugriff erfolgt nur über eine offizielle, dokumentierte Schnittstelle. Eine Datenstruktur ist über ihr Verhalten definiert, nicht über ihre internen Daten. Das Innenleben kann geändert werden, ohne den aufrufenden Code anzupassen.
- **Code-Wiederverwendung:** über Vererbung (neuer Typ als Erweiterung eines bestehenden) und Templates (parametrierter Code mit Typen als Parameter).

**Denkweise:** Prozedural ist ablauforientiert und top-down; OOP ist datenorientiert und bottom-up.

Daten sind „aktiv“ – sie führen Operationen auf sich selbst aus. Statt `drawButton(okButton)` schreibt man `okButton.draw()`.

## 2. Die vier Grundpfeiler der OOP

Eine sehr häufige Einstiegsfrage. Vier Säulen merken:

| Säule       | Bedeutung                                            | In C++ durch                 |
|-------------|------------------------------------------------------|------------------------------|
| Abstraktion | Wesentliches modellieren, Details weglassen          | Klassen, abstrakte Klassen   |
| Kapselung   | Daten verstecken, Zugriff nur über die Schnittstelle | private / protected, get/set |
| Vererbung   | Neue Klasse erweitert eine bestehende                | class B : public A           |
| Polymorphie | Gleiche Schnittstelle, unterschiedliches Verhalten   | virtuelle Methoden (virtual) |

## 3. Grundbegriffe

- **Klasse:** Beschreibung gleichartiger Objekte (Schnittstelle + Innenleben, Daten + Code). Technisch ein struct-Typ, erweitert um Funktionen. Eine Klasse ist ein Typ (Schablone/Prototyp) und enthält selbst keine konkreten Werte.
- **Objekt:** eine Instanz einer Klasse – ein konkreter Wert im Speicher. Von einer Klasse kann es beliebig viele (oder keine) Objekte geben.
- **Methode (Member-Funktion):** eine zu einer Klasse gehörige Funktion.
- **Member-Variable (Attribut):** ein zu einer Klasse gehöriges Datenelement.

Beispiel: Klasse Button → Objekte „Speichern“, „Abbrechen“, „OK“.

## 4. Klassen und Objekte in C++

- Schlüsselwort `class` statt `struct` (`struct` kann fast dasselbe).
- **Zugriffsregelung:** `public`: (im ganzen Programm sichtbar), `private`: (nur eigene Methoden), `protected`: (zusätzlich abgeleitete Klassen).
- **Unterschied `struct/class`:** In `struct` ist per Default alles `public`, in `class` per Default alles `private`.
- **Stil:** zuerst `public` (Schnittstelle), dann `private` (Innenleben).

```
class Point {  
public:                // öffentliche Schnittstelle  
    Point(int x, int y); // Konstruktor  
    int getX() const;   // get-Methode (const!)  
    void setX(int x);  
private:              // geheimes Innenleben  
    int mX, mY;        // Member-Variablen  
};
```

**Methoden-Definition:** entweder direkt im class (dann automatisch inline, nur für kurze Methoden) oder außerhalb mit Klassennamen und Scope-Operator: `int Point::getX() const { return mX; }`

**Objekt anlegen:** `Point a(3, 4);` am Stack oder dynamisch `Point *p = new Point(3, 4);` am Heap.

## 5. Konstruktoren (Ctor)

Methode mit dem Namen der Klasse, kein Returntyp (auch nicht `void`). Wird automatisch beim Anlegen eines Objekts aufgerufen (lokale Variable, `new`, `by-Value`-Übergabe, ...) und dient zur Initialisierung: Ein Objekt soll immer gleich funktionsfähig „geboren“ werden.

### Arten von Konstruktoren

- **Standard-Konstruktor (Default):** ohne Parameter. Wird automatisch erzeugt, wenn die Klasse gar keinen expliziten Konstruktor hat. Sobald man irgendeinen eigenen Konstruktor definiert, verschwindet der automatische Standard-Konstruktor.
- **Copy-Konstruktor:** bekommt eine (`const`-)Referenz auf ein Objekt der eigenen Klasse. Automatisch = bitweise Kopie. Ein eigener Copy-Konstruktor ist nötig, wenn Member Pointer sind – sonst zeigen Original und Kopie auf dieselben Daten (flache Kopie).
- **Typumwandlungs-Konstruktor:** ein Parameter anderen Typs. Mit `explicit` davor nur explizite Aufrufe (keine impliziten Umwandlungen).

**Wann läuft der Copy-Konstruktor?** Bei Initialisierungs-Zuweisung und bei `Call/Return by Value` – nicht bei normaler Zuweisung (da läuft der `=`-Operator).

**„Verhinderte“ Konstruktoren:** Konstruktoren `private` machen → Objekte nur von der eigenen Klasse erzeugbar (Singleton). Mit `= delete` (C++11) ganz unterbinden (oft beim Copy-Konstruktor).

## 6. Destruktoren (Dtor)

- Methode mit dem Klassennamen und `~` davor, kein Returntyp, keine Parameter.
- Nur ein Destruktor pro Klasse (anders als bei Konstruktoren).
- Wird automatisch beim Abbau des Objekts aufgerufen: Blockende, Programmende, `delete`, Vernichtung temporärer Objekte.

- **Aufgabe:** Aufräumarbeiten – vor allem dynamisch angelegten Speicher von Pointer-Membren freigeben (Gegenstück zu new → delete). Fehlt das, entsteht ein Speicherleck.

## 7. Initialisierungsliste, this, const-Methoden

**Initialisierungsliste:** steht im Konstruktorkopf nach der ) und vor der {:

```
Point(int x, int y) : mX(x), mY(y) { /* ... */ }
```

- Member werden in Reihenfolge ihrer Deklaration initialisiert (nicht in Reihenfolge der Liste).
- Notwendig für const-Member und Referenz-Member (müssen sofort initialisiert werden).
- Effizienter als Zuweisungen im Rumpf (kein „erst leer anlegen, dann füllen“).

**this:** nur innerhalb von Methoden definiert; Pointer auf das eigene Objekt (der versteckte „nullte Parameter“). Nötig z. B. bei Namenskonflikt: this->mX.

**const-Methode:** int getX() const; → ändert das Objekt nicht. Auf const-Objekte dürfen nur const-Methoden angewendet werden.

**get/set-Konvention:** Member sind nicht public. Für lesbare Member gibt es eine get...-Methode (meist inline und const), für änderbare eine set...-Methode.

## 8. Statische Member und Methoden

- **Statisches Member:** existiert einmal pro Klasse, nicht pro Objekt – alle Objekte teilen sich den Wert (z. B. Objektzähler). Muss außerhalb der Klasse definiert werden: int Point::counter = 0;
- **Statische Methode:** wird für die Klasse aufgerufen (Point::statMeth()), nicht für ein Objekt. this existiert nicht; Zugriff nur auf statische Member.

## 9. Heap vs. Stack

Wurde in der Prüfung beim Klassenaufbau explizit verlangt.

- **Stack:** automatischer Speicher für lokale Variablen; schnell; wird beim Verlassen des Blocks automatisch freigegeben; begrenzte Größe. Der Destruktor läuft hier automatisch. Beispiel: Point p(1,2);
- **Heap:** dynamischer Speicher, manuell mit new angelegt und mit delete freizugeben; flexibel und groß; bei vergessenen delete → Speicherleck. Beispiel: Point \*p = new Point(1,2); ... delete p;

## 10. Dynamische Speicherverwaltung: new und delete

- **new** legt Speicher dynamisch an, ruft (bei Objekten) den Konstruktor auf und liefert einen Pointer.

```
int *p = new int;           // einzelner int
MyClass *o = new MyClass(10); // Objekt + Konstruktor
int *a = new int[10];       // Array
```

- **delete** ruft den Destruktor auf und gibt den Speicher frei. Zwei passende Varianten: delete p; (einzelnes Objekt) und delete [] a; (Array).
- delete mit dem NULL-Pointer ist erlaubt (tut nichts). Hat eine Klasse keinen Standard-Konstruktor, kann kein Array von Objekten angelegt werden.

## 11. Vererbung

Die abgeleitete Klasse (derived class) erweitert eine Basisklasse (base class) und erbt alle deren Member und Methoden.

```
class Button : public DialogElement { /* ... */ };
```

- Man erbt immer alles – Geerbtes kann nicht versteckt oder gelöscht werden.
- Konstruktoren und Destruktoren werden nicht vererbt – sie existieren in jeder Klasse separat.
- **protected**: nach außen unsichtbar (wie private), aber für abgeleitete Klassen voll zugänglich.

### Design-Faustregel

„X ist ein Y“ (Rechteck ist ein geometrisches Objekt) → Vererbung. „X hat ein Y“ (Rechteck hat eine Farbe) → Member-Variable.

## 12. Überschreiben und virtuelle Methoden (Polymorphie)

**Überschreiben**: Eine abgeleitete Klasse gibt eigenen Code für eine schon vorhandene Methode an. Es läuft immer nur der zutreffendste Code (gesucht von der eigenen Klasse aufwärts).

**Statische vs. dynamische Bindung (der zentrale Punkt)**:

```
myBaseClass *p = new mySubClass(...);  
p->myFunc(); // Welche myFunc läuft?
```

- **Mit virtual** in der Basisklasse → dynamische Bindung: zur Laufzeit wird die Methode der tatsächlichen Klasse des Objekts aufgerufen (mySubClass). Das erwartet man meist (in Java/C# immer so).
- **Ohne virtual** → statische Bindung: zur Compilezeit nach dem deklarierten Typ von p → myBaseClass.
- virtual vererbt sich automatisch. C++ nutzt standardmäßig die billigere statische Bindung, weil virtual Laufzeit/Speicher kostet (vtable-Zeiger pro Objekt).

## 13. Abstrakte Klassen und rein virtuelle Methoden

**Rein virtuelle Methode (pure virtual)**: in der Basisklasse deklariert, aber nicht implementiert, weil die Umsetzung je abgeleiteter Klasse anders ist:

```
virtual void draw() = 0; // = 0 -> rein virtuell
```

- Eine Klasse ist automatisch abstrakt, sobald sie mindestens eine rein virtuelle Methode hat.
- Von einer abstrakten Klasse können keine Objekte angelegt werden – nur Pointer/Referenzen darauf (die auf Objekte abgeleiteter Klassen zeigen).
- Eine abgeleitete Klasse ist erst dann nicht mehr abstrakt, wenn sie alle geerbten rein virtuellen Methoden implementiert.

### Virtueller Destruktor

Sobald eine Klasse virtuelle Methoden hat, sollte der Destruktor virtual sein (Faustregel ~95 %), sonst werden beim delete über einen Basisklassen-Pointer nicht alle Destruktoren ausgeführt → Speicherleck.

## 14. Typumwandlung zwischen verwandten Klassen

- **static\_cast zur Basisklasse hin:** sicher, immer korrekt.
- **static\_cast zur abgeleiteten Klasse hin:** erlaubt, aber unsicher – keine Laufzeitprüfung.
- **dynamic\_cast:** prüft zur Laufzeit; bei Misserfolg NULL (bei Referenzen Exception). Funktioniert nur bei Klassen mit mindestens einer virtuellen Methode (RTTI), daher langsamer.

## 15. Aufteilung in Dateien

- Pro Klasse ein .h- und ein .cpp-File.
- **.h:** Klassendeklaration + alle inline-Methoden + Templates (öffentliche Schnittstelle).
- **.cpp:** Methoden-Definitionen + Definition statischer Member.
- Header immer mit Include-Guard schützen: #ifndef / #define / #endif.

## 16. Unterschied C und C++

C++ ist eine (fast) objektorientierte Erweiterung von C – fast jedes C-Programm ist gültiges C++.  
Wesentliche Erweiterungen:

- Klassen (class) mit Methoden, Kapselung, Vererbung, Polymorphie.
- new / delete statt malloc / free; Referenzen (&) zusätzlich zu Pointern.
- Überladen von Funktionen/Operatoren, Default-Parameter, Templates, Namespaces, Exceptions.
- C++-I/O mit cin/cout; bool ohne Header; nullptr statt NULL.

**Kurzform:** C ist prozedural, C++ ist prozedural plus objektorientiert (und mehr).

## 17. Templates

Parametrierter Code: Datentypen werden als Parameter übergeben, sodass derselbe Code mit verschiedenen Typen funktioniert (vermeidet Duplikate). Die STL (vector, map, ...) basiert darauf.

```
template <typename T>
T max(T a, T b) { return (a > b) ? a : b; }
```

## 18. Geschichte und Sprachen (kurz)

- C++ ab 1979 von Bjarne Stroustrup (AT&T Bell Labs) als Erweiterung von C; 1985 veröffentlicht, 1998 ISO-Standard, 2011 stark erweitert (C++11), danach alle 3 Jahre eine neue Version.
- **Rein objektorientiert** („alles ist ein Objekt“): Smalltalk, Ruby, Eiffel.
- **Gemischt objektorientiert:** C++, Java, C#, Python, Objective-C.
- **Nachträglich erweitert:** PHP, Perl, Visual Basic.

## Teil B – Prüfungsfragen mit Modellantworten

---

Diese Fragen stammen aus den tatsächlich protokollierten OOP-Fragen von Herrn Heinze und den Externen. Kurze, laut sprechbare Antworten zum Abfragen.

### F1: Erklären Sie OOP – was sind die Kernkonzepte / Säulen?

OOP ist eine datenorientierte Art zu programmieren: Daten und die Funktionen, die auf ihnen arbeiten, werden zu Objekten zusammengefasst. Die vier Säulen:

1. **Abstraktion** – nur das Wesentliche modellieren, Details weglassen.
2. **Kapselung** – Daten privat, Zugriff nur über öffentliche Methoden; Innenleben austauschbar.
3. **Vererbung** – eine Klasse erweitert eine bestehende und übernimmt deren Member/Methoden.
4. **Polymorphie** – gleiche Schnittstelle, unterschiedliches Verhalten (in C++ über virtual).

**Anwendung:** große Systeme strukturieren, Code wiederverwenden, Realität modellieren (GUIs, Simulationen ...).

### F2: Unterschied zwischen Klasse und Objekt?

Eine Klasse ist der Bauplan / die Schablone (ein Typ) – sie beschreibt Daten und Methoden, hat aber keine konkreten Werte. Ein Objekt ist eine Instanz dieser Klasse – ein konkreter Wert im Speicher. Von einer Klasse Button kann es viele Objekte geben („OK“, „Abbrechen“ ...).

### F3: Wie ist eine Klasse aufgebaut? (Lieblingsfrage)

Eine Klasse besteht aus:

- **Member-Variablen (Attribute)** – die Daten, üblicherweise private.
- **Methoden** – das Verhalten.
- **Konstruktor** – initialisiert das Objekt beim Anlegen.
- **Destruktor** – räumt beim Abbau auf.
- **Zugriffsregelung** – public (Schnittstelle), private (Innenleben), protected (auch abgeleitete Klassen).
- **get-/set-Methoden** – kontrollierter Zugriff auf private Member.

```
class Konto {
public:                                // Schnittstelle
    Konto(double start);              // Konstruktor
    ~Konto();                          // Destruktor
    double getStand() const;          // getter (const!)
    void einzahlen(double betrag);     // Methode
private:                              // Innenleben (gekapselt)
    double mStand;
};
```



## F4: Konstruktor und Destruktor (besonders Destruktor)

**Konstruktor (Ctor):** Methode mit dem Namen der Klasse, kein Returntyp. Wird automatisch beim Anlegen aufgerufen und initialisiert das Objekt. Es kann mehrere geben (Standard-, Copy-, Typumwandlungs-Konstruktor).

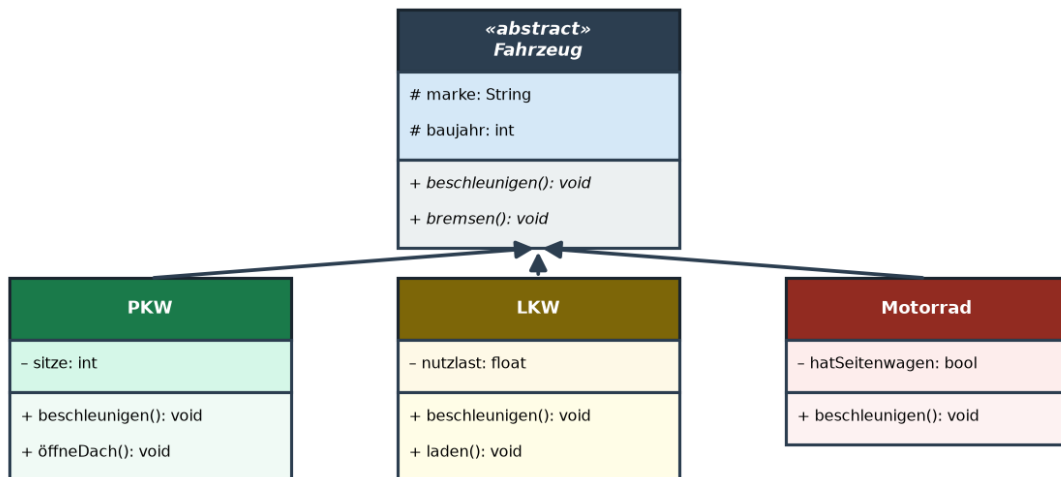
**Destruktor (Dtor):** ~Klassenname(), kein Returntyp, keine Parameter, nur einer pro Klasse. Wird automatisch beim Abbau aufgerufen (Blockende, delete, Programmende). Aufgabe: aufräumen, vor allem dynamischen Speicher freigeben. Fehlt das bei Pointer-Membnern → Speicherleck.

## F5: Erklären Sie Vererbung

Eine neue Klasse (abgeleitete Klasse) wird als Erweiterung einer Basisklasse definiert und erbt alle deren Member und Methoden.

```
class Fahrzeug { /* Geschwindigkeit, fahren() ... */ };  
class PKW : public Fahrzeug { /* zusätzlich: Kofferraum ... */ };
```

UML-Klassendiagramm - Vererbung



«abstract» = abstrakte Klasse | # = protected | - = private | + = public

Abb.: UML-Klassendiagramm – Vererbung

- Man erbt immer alles – Geerbtes kann nicht gelöscht werden.
- Geerbte Methoden dürfen überschrieben werden.
- **Vorteil:** Code-Wiederverwendung – nur das Neue/Andere wird programmiert.
- **Faustregel:** „PKW ist ein Fahrzeug“ → Vererbung passt.

## F6: Was ist Aggregation? (und die anderen UML-Beziehungen)

Aggregation ist eine „hat-ein / besteht-aus“-Beziehung (Teil-Ganzes), bei der die Teile auch unabhängig existieren können. Beispiel: Ein Team hat Spieler – löst man das Team auf, existieren die Spieler weiter.

| Beziehung   | Bedeutung                      | Beispiel           | UML-Pfeil                      |
|-------------|--------------------------------|--------------------|--------------------------------|
| Assoziation | kennt / nutzt                  | Kunde – Bestellung | einfache Linie / Pfeil         |
| Aggregation | hat ein, Teile überleben       | Team – Spieler     | leere Raute am Ganzen          |
| Komposition | besteht aus, Teile sterben mit | Haus – Raum        | gefüllte Raute am Ganzen       |
| Vererbung   | ist ein                        | PKW – Fahrzeug     | leerer Dreieckspfeil zur Basis |

**Merksatz**  
Aggregation = lose („hat“), Komposition = stark („besteht aus“, Teile sterben mit), Vererbung = „ist ein“.

## F7: Zeichnen Sie ein Klassendiagramm

Ein UML-Klassendiagramm zeigt Klassen als Kästchen mit drei Bereichen: Klassenname, Attribute, Methoden. Sichtbarkeit: – private, + public, # protected.

```
+-----+
| Konto          | <- Klassenname
+-----+
| - stand : double | <- Attribute
+-----+
| + getStand() : double | <- Methoden
| + einzahlen(b:double) |
+-----+
```

Die Beziehungen zwischen den Klassen werden mit den Pfeilen aus F6 eingezeichnet (Vererbung = Dreieckspfeil, Aggregation = leere Raute, Komposition = gefüllte Raute).

## F8: Schreiben Sie eine Klasse an (mit public, private, protected)

```
class Person {
public:                                // überall sichtbar
    Person(std::string name, int alter) // Konstruktor (Init-Liste)
        : mName(name), mAlter(alter) {}
    ~Person() {}                       // Destruktor
    std::string getName() const { return mName; } // getter
    void setAlter(int alter) { mAlter = alter; } // setter
protected:                            // eigene + abgeleitete Klassen
    int mAlter;
private:                               // nur eigene Klasse
    std::string mName;
};
```

**Beim Anschreiben laut erklären:**

- **public** = öffentliche Schnittstelle, von überall aufrufbar.
- **private** = nur Methoden der eigenen Klasse (Kapselung).
- **protected** = wie private, aber abgeleitete Klassen dürfen zugreifen.
- Konstruktor initialisiert per Initialisierungsliste, getter ist const.

## F9: Unterschied C und C++

C++ ist eine (fast) objektorientierte Erweiterung von C. Wesentliche Erweiterungen: Klassen (OOP), new/delete, Referenzen, Überladen, Default-Parameter, Templates, Namespaces, Exceptions, C++-I/O. Kurz: C ist prozedural, C++ ist prozedural plus objektorientiert.

## F10: Was sind Templates?

Parametrierter Code – Datentypen als Parameter, damit derselbe Code mit verschiedenen Typen funktioniert (keine Duplikate). Die STL basiert darauf.

```
template <typename T>
T max(T a, T b) { return (a > b) ? a : b; }
```

## F11: Heap vs. Stack

- **Stack:** automatischer Speicher für lokale Variablen; schnell; automatische Freigabe beim Blockende; begrenzt. Konto k(100);
- **Heap:** dynamisch mit new angelegt, mit delete freizugeben; flexibel/groß; vergessenes delete → Speicherleck. Konto \*k = new Konto(100); ... delete k;

Konstruktor läuft bei beidem; der Destruktor läuft am Stack automatisch, am Heap erst bei delete.

## F12: Sechs Programmiersprachen – objektorientiert? Compiler oder Interpreter?

| Sprache    | Objektorientiert?     | Übersetzung                   |
|------------|-----------------------|-------------------------------|
| C          | nein (prozedural)     | Compiler                      |
| C++        | ja (auch prozedural)  | Compiler                      |
| Java       | ja                    | Mischform: Bytecode + JVM/JIT |
| Python     | ja                    | Interpreter (Bytecode + VM)   |
| C#         | ja                    | Mischform (CIL + JIT)         |
| JavaScript | ja (prototyp-basiert) | Interpreter / JIT             |

### Compiler vs. Interpreter (kommt fast sicher dazu):

- **Compiler:** übersetzt das ganze Programm vorab in Maschinencode → schnell zur Laufzeit, plattformabhängig (C, C++).
- **Interpreter:** führt den Code Zeile für Zeile zur Laufzeit aus → flexibler/portabler, aber langsamer (Python).
- **Java (Mischform):** Quellcode → Bytecode (compiliert) → von der JVM interpretiert; häufige Stellen per JIT zur Laufzeit in Maschinencode übersetzt.

## F13: Wofür / warum benutzt man OOP überhaupt?

Um große Software beherrschbar zu machen: bessere Strukturierung (Funktionen gehören zu ihren Daten), Kapselung (klare Schnittstellen, austauschbares Innenleben), Wiederverwendung (Vererbung, Templates) und realitätsnahe Modellierung (zusammenspielende Objekte). Erleichtert Teamarbeit, Wartung und Erweiterung.

## Schnell-Check vor der Prüfung

---

- **4 Säulen:** Abstraktion, Kapselung, Vererbung, Polymorphie.
- Klasse blind anschreiben können (Konstruktor, Destruktor, get/set, public/private/protected).
- **Destruktor-Zweck:** Aufräumen / Speicher freigeben.
- **UML-Pfeile:** Vererbung = Dreieck, Aggregation = leere Raute, Komposition = volle Raute.
- „ist ein“ = Vererbung, „hat ein“ = Aggregation/Member.
- **Heap** (new/delete) vs. **Stack** (automatisch).
- **C vs. C++:** C++ = C + OOP + Templates/Referenzen/Exceptions.
- **virtual** = dynamische Bindung (richtet sich nach der tatsächlichen Klasse des Objekts).